



DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES

■ Implementación con Keras

TLY1102

Decisiones de diseño de una red Fully Connected

ARQUITECTURA

Capas ocultas

Más capas permiten composiciones más complejas; también aumentan riesgo de overfitting.

Neuronas por capa

Más neuronas aumentan capacidad. Pocas neuronas pueden provocar underfitting.

Activaciones

ReLU en capas ocultas;
sigmoid/softmax/lineal según la salida.



Lectura de la arquitectura

La red transforma los datos capa a capa. Cada capa oculta aprende representaciones intermedias y la última capa traduce esa representación a probabilidad, clase o valor continuo.

```

Sequential([
  Dense(16, activation='relu'),
  Dense(8, activation='relu'),
  Dense(1, activation='sigmoid')
])
  
```

Keras: API secuencial

IMPLEMENTACIÓN

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

modelo = Sequential([
    Dense(16, activation='relu', input_shape=(4,)),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

- `Sequential` organiza capas como una pila lineal.
- La salida de una capa entra directamente a la siguiente.
- `Dense` significa capa completamente conectada.
- `input\_shape` define cuántas variables recibe la red.
- La última capa depende del tipo de problema.

En la imagen...

Entrada de 4 variables → 16 neuronas ReLU → 8 neuronas ReLU → 1 probabilidad sigmoid.

Compilar el modelo: cómo aprende y qué miramos

KERAS COMPILE

```
modelo.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

Optimizer

Define cómo se actualizan los pesos. Ejemplos: SGD, Adam.

Loss

Función que el entrenamiento intenta minimizar. Guía el aprendizaje.

Metrics

Indicadores para interpretar rendimiento. No necesariamente se optimizan directamente.

Tener en cuenta

La loss mueve los pesos; la métrica nos ayuda a leer si el modelo sirve para el objetivo.

Entrenamiento: epochs, batch size y validación

FIT

```
historia = modelo.fit(  
    X_train, y_train,  
    epochs=80,  
    batch_size=32,  
    validation_split=0.2  
)
```

Epoch

Una pasada completa por los datos de entrenamiento.

Batch size

Número de ejemplos usados antes de actualizar los pesos.

Validation split

Parte del entrenamiento reservada para monitorear generalización.

Objeto historia

Guarda loss y métricas por epoch para graficar curvas de aprendizaje.

Modelo clasificador en Keras

CÓDIGO BASE

```
def build_classifier(hidden_layers=(8, 8)):
    model = Sequential()
    model.add(Dense(hidden_layers[0], activation='relu', input_shape=(2,)))

    for units in hidden_layers[1:]:
        model.add(Dense(units, activation='relu'))

    model.add(Dense(1, activation='sigmoid'))
    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model
```

Qué se modifica en clase

`hidden\_layers=(2,)`, `(8,8)`, `(32,32,16)` para observar capacidad del modelo.

Qué se mantiene

Salida sigmoid y binary_crossentropy, porque el problema sigue siendo clasificación binaria.

Qué se mide

Accuracy, precision, recall, F1-score, matriz de confusión y curvas ROC / PR.

Laboratorio guiado: qué modificar

Arquitectura

Cambiar `hidden_layers`: (2,), (8,8), (32,32,16), (64,64,32).

Optimizador

Comparar `adam` y `sgd`: velocidad de convergencia y sensibilidad.

Learning rate

Probar 0.1, 0.01, 0.001 y 0.0001. Observar curvas de pérdida.

Métricas

Cambiar el foco: F1 en clasificación, R^2 /RMSE en regresión.

- ¿La arquitectura más grande siempre gana?
- ¿Qué pasa cuando el learning rate es demasiado alto?
- ¿Cuándo preferir recall sobre precision?
- ¿Por qué R^2 puede ser negativo?
- ¿Qué gráfico usarías para detectar errores sistemáticos?

Errores frecuentes y cómo corregirlos

Usar accuracy en regresión

Corregir usando MAE, RMSE y R^2 .

Salida sigmoid para regresión

Usar `Dense(1)` lineal si se predice un valor continuo.

No escalar variables

Aplicar StandardScaler o normalización para estabilizar el entrenamiento.

Evaluar solo training

Separar train, validation y test para estimar generalización.

Mirar solo una métrica

Combinar métricas y visualizaciones: matriz, curvas, residuos, real vs predicho.

Regla mínima

- 1) Elegir salida según problema
- 2) Elegir loss coherente
- 3) Entrenar con validación
- 4) Evaluar en test
- 5) Graficar errores

Cierre: lógica que se repite en todo Deep Learning



Clasificación

Salida sigmoid/softmax · Cross-entropy · Matriz de confusión · Precision/Recall/F1.

Regresión

Salida lineal · MSE/MAE · MAE/RMSE/R² · Real vs predicho · Residuos.

Mensaje final

El objetivo no es que el modelo “entrene”, sino que aprenda una relación útil y generalizable, medida con métricas coherentes con el problema.

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO



NIVEL DE EXCELENCIA

HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la calidad / Vinculación con el Medio / Investigación, creación y/o innovación